Sales Analysis — Course-End Project

This notebook follows only the tasks listed in the supplied problem statement: data preparation, normalization, overall daily trends, monthly data and descriptive statistics, monthly boxplots and plots, comprehensive snapshot, statewise sales, groupwise analysis, and timewise analysis.

1. Prepare Data for Analysis

In [3]:
```
!mamba install pandas numpy matplotlib seaborn scikit-learn
```

```
mambajs 0.21.4

Specs: xeus-python, numpy, matplotlib, pillow, ipywidgets>=8.1.6, ipyleaflet, sci
py, pandas, seaborn, scikit-learn
Channels: emscripten-forge-4x, conda-forge

Solving environment...
Solving took 2.175 seconds
  Name                Version    Build                Channel
------------------------------------------------------------------------------
+ brotli-python       1.2.0      py313ha26e73d_2      emscripten-forge-4x
+ certifi             2026.7.22  pyhd8ed1ab_0         conda-forge
+ charset-normalizer  3.5.1      pyhd8ed1ab_0         conda-forge
+ idna                3.20       pyh5ded981_0         conda-forge
+ joblib              1.6.0      py313h57e9779_0      emscripten-forge-4x
+ narwhals            2.27.0     pyh5ded981_0         conda-forge
+ pandas              3.0.6      np23py313h124dbc7_0  emscripten-forge-4x
+ patsy               1.0.3      py313h1804a44_0      emscripten-forge-4x
+ pysocks             1.7.1      py313h1804a44_3      emscripten-forge-4x
+ python-tzdata       2026.5     pyh5ded981_0         conda-forge
+ requests            2.34.2     pyhcf101f3_0         conda-forge
+ scikit-learn        1.9.0      np23py313hb9816c2_0  emscripten-forge-4x
+ seaborn             0.13.2     hd8ed1ab_3           conda-forge
+ seaborn-base        0.13.2     pyhd8ed1ab_3         conda-forge
+ setuptools          84.0.0     pyh332efcf_0         conda-forge
+ statsmodels         0.14.6     np23py313hd8db738_2  emscripten-forge-4x
+ threadpoolctl       3.7.0      pyhc455866_0         conda-forge
+ urllib3             2.2.2      pyhd8ed1ab_0         conda-forge
- pip                 26.2.1     pyh145f28c_0         conda-forge
```

In [4]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (12, 5)

df = pd.read_csv("Sales.csv")
df["Date"] = pd.to_datetime(df["Date"], format="%d-%b-%y", errors="coerce")

print("DataFrame:")
```

```
display(df)
print("Dimensions (rows, columns):", df.shape)
print("\nMissing values by column:")
display(df.isnull().sum())
print("\nTotal missing values:", df.isnull().sum().sum())
print("\nData types:")
display(df.dtypes)
```

DataFrame:

	Date	Time	State	Group	Unit	Sales
0	2020-10-01	Morning	WA	Kids	8	20000
1	2020-10-01	Morning	WA	Men	8	20000
2	2020-10-01	Morning	WA	Women	4	10000
3	2020-10-01	Morning	WA	Seniors	15	37500
4	2020-10-01	Afternoon	WA	Kids	3	7500
...	...	...	...	...	...	...
7555	2020-12-30	Afternoon	TX	Seniors	14	35000
7556	2020-12-30	Evening	TX	Kids	15	37500
7557	2020-12-30	Evening	TX	Men	15	37500
7558	2020-12-30	Evening	TX	Women	11	27500
7559	2020-12-30	Evening	TX	Seniors	13	32500

7560 rows × 6 columns

```
Dimensions (rows, columns): (7560, 6)

Missing values by column:
Date     0
Time     0
State    0
Group    0
Unit     0
Sales    0
dtype: int64
Total missing values: 0

Data types:
Date     datetime64[us]
Time                str
State               str
Group               str
Unit              int64
Sales             int64
dtype: object
```

2. Normalize Data for Analysis

In [5]:
```
df_dataonly = df[["Unit", "Sales"]].copy()
normalize = MinMaxScaler()
normalize_data = normalize.fit_transform(df_dataonly)
```

```
print("Shape of normalize_data:", normalize_data.shape)
print("Minimum values (Unit, Sales):", normalize_data.min(axis=0))
print("Maximum values (Unit, Sales):", normalize_data.max(axis=0))
```

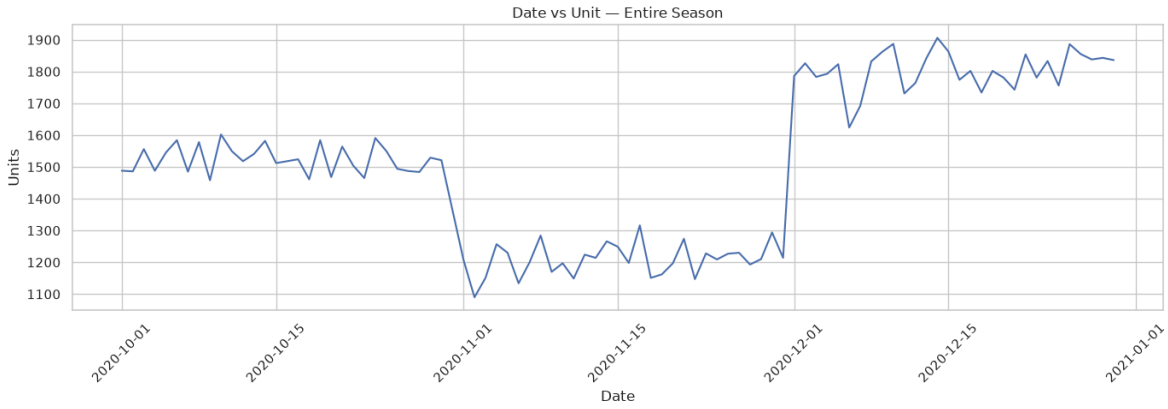
```
Shape of normalize_data: (7560, 2)
Minimum values (Unit, Sales): [0. 0.]
Maximum values (Unit, Sales): [1. 1.]
```

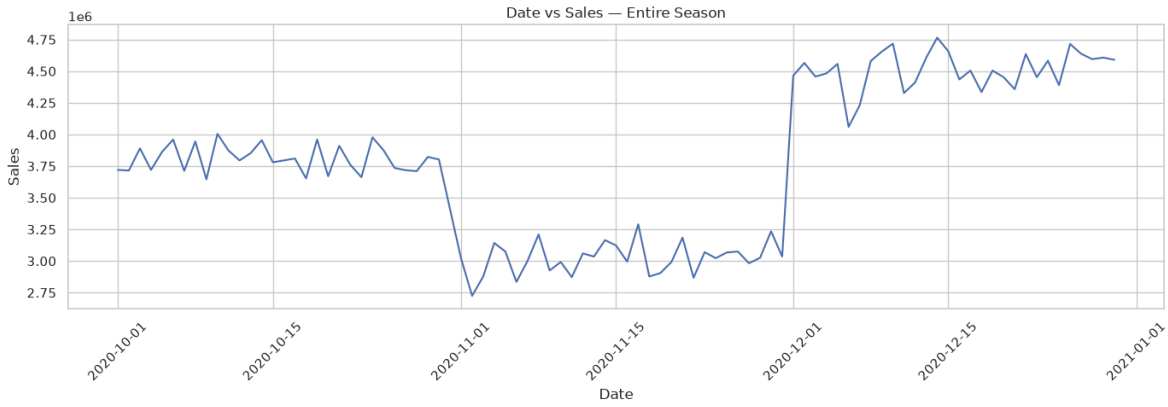
3. Visualize Overall Trends

In [6]:
```
daily = df.groupby("Date", as_index=False).agg({"Unit": "sum", "Sales": "sum"}).

plt.figure(figsize=(14, 5))
plt.plot(daily["Date"], daily["Unit"])
plt.title("Date vs Unit — Entire Season")
plt.xlabel("Date")
plt.ylabel("Units")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

plt.figure(figsize=(14, 5))
plt.plot(daily["Date"], daily["Sales"])
plt.title("Date vs Sales — Entire Season")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```





4. Analyze Monthly Data

In [7]:
```
df_oct = df.loc[(df["Date"] >= "2020-10-01") & (df["Date"] < "2020-11-01")].copy
df_nov = df.loc[(df["Date"] >= "2020-11-01") & (df["Date"] < "2020-12-01")].copy
df_dec = df.loc[(df["Date"] >= "2020-12-01") & (df["Date"] < "2021-01-01")].copy

print("October data")
display(df_oct)
print("November data")
display(df_nov)
print("December data")
display(df_dec)
```

```
October data
```

	Date	Time	State	Group	Unit	Sales
0	2020-10-01	Morning	WA	Kids	8	20000
1	2020-10-01	Morning	WA	Men	8	20000
2	2020-10-01	Morning	WA	Women	4	10000
3	2020-10-01	Morning	WA	Seniors	15	37500
4	2020-10-01	Afternoon	WA	Kids	3	7500
...	...	...	...	...	...	...
2515	2020-10-30	Afternoon	TX	Seniors	12	30000
2516	2020-10-30	Evening	TX	Kids	15	37500
2517	2020-10-30	Evening	TX	Men	7	17500
2518	2020-10-30	Evening	TX	Women	10	25000
2519	2020-10-30	Evening	TX	Seniors	5	12500

2520 rows × 6 columns

```
November data
```

	Date	Time	State	Group	Unit	Sales
2520	2020-11-01	Morning	WA	Kids	4	10000
2521	2020-11-01	Morning	WA	Men	6	15000
2522	2020-11-01	Morning	WA	Women	4	10000
2523	2020-11-01	Morning	WA	Seniors	7	17500
2524	2020-11-01	Afternoon	WA	Kids	8	20000
...	...	...	...	...	...	...
5035	2020-11-30	Afternoon	TX	Seniors	9	22500
5036	2020-11-30	Evening	TX	Kids	9	22500
5037	2020-11-30	Evening	TX	Men	10	25000
5038	2020-11-30	Evening	TX	Women	9	22500
5039	2020-11-30	Evening	TX	Seniors	2	5000

2520 rows × 6 columns

```
December data
```

	Date	Time	State	Group	Unit	Sales
5040	2020-12-01	Morning	WA	Kids	9	22500
5041	2020-12-01	Morning	WA	Men	11	27500
5042	2020-12-01	Morning	WA	Women	15	37500
5043	2020-12-01	Morning	WA	Seniors	13	32500
5044	2020-12-01	Afternoon	WA	Kids	13	32500
...	...	...	...	...	...	...
7555	2020-12-30	Afternoon	TX	Seniors	14	35000
7556	2020-12-30	Evening	TX	Kids	15	37500
7557	2020-12-30	Evening	TX	Men	15	37500
7558	2020-12-30	Evening	TX	Women	11	27500
7559	2020-12-30	Evening	TX	Seniors	13	32500

2520 rows × 6 columns

5. Describe Data

In [8]:
```
print("Full three-month period:")
display(df[["Unit", "Sales"]].describe())
print("October:")
display(df_oct[["Unit", "Sales"]].describe())
print("November:")
display(df_nov[["Unit", "Sales"]].describe())
```

```
print("December:")
display(df_dec[["Unit", "Sales"]].describe())
```

```
Full three-month period:
```

	Unit	Sales
count	7560.000000	7560.000000
mean	18.005423	45013.558201
std	12.901403	32253.506944
min	2.000000	5000.000000
25%	8.000000	20000.000000
50%	14.000000	35000.000000
75%	26.000000	65000.000000
max	65.000000	162500.000000

```
October:
```

	Unit	Sales
count	2520.000000	2520.000000
mean	18.141270	45353.174603
std	11.944521	29861.302213
min	3.000000	7500.000000
25%	9.000000	22500.000000
50%	14.000000	35000.000000
75%	27.000000	67500.000000
max	50.000000	125000.000000

```
November:
```

	Unit	Sales
count	2520.000000	2520.000000
mean	14.394048	35985.119048
std	10.946470	27366.175823
min	2.000000	5000.000000
25%	6.000000	15000.000000
50%	10.000000	25000.000000
75%	22.000000	55000.000000
max	45.000000	112500.000000

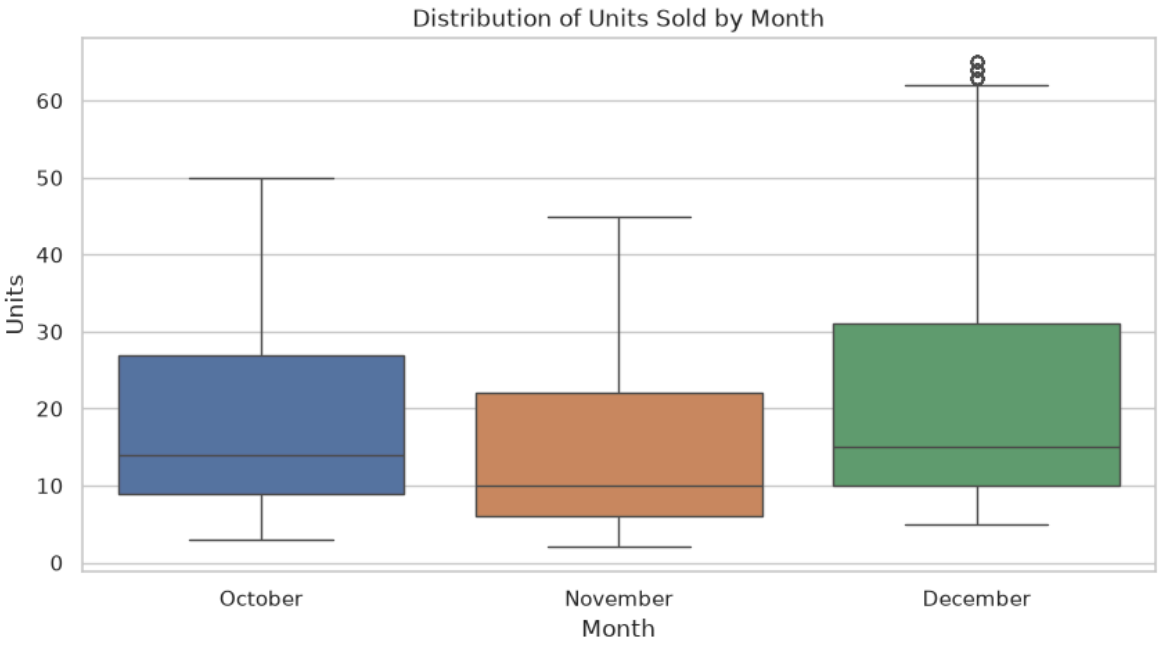
```
December:
```

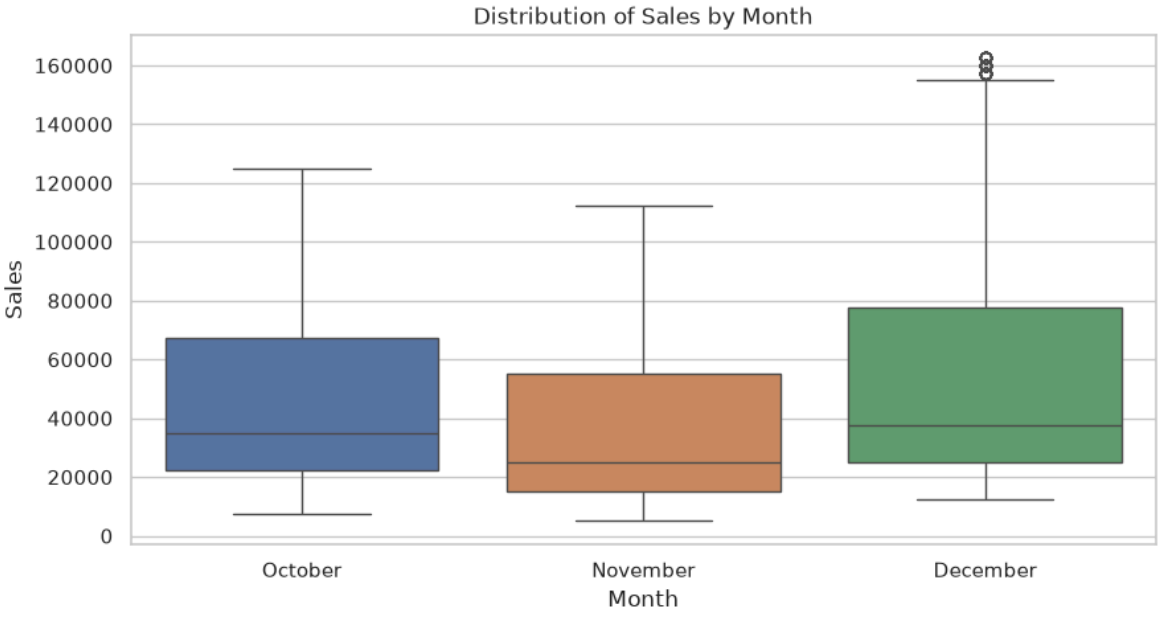
	Unit	Sales
count	2520.000000	2520.000000
mean	21.480952	53702.380952
std	14.554181	36385.451298
min	5.000000	12500.000000
25%	10.000000	25000.000000
50%	15.000000	37500.000000
75%	31.000000	77500.000000
max	65.000000	162500.000000

6. Analyze Unit Data and Sales Distribution

In [9]:
```
unit_boxplot_data = pd.DataFrame({
    "October": df_oct["Unit"].reset_index(drop=True),
    "November": df_nov["Unit"].reset_index(drop=True),
    "December": df_dec["Unit"].reset_index(drop=True)
})
plt.figure(figsize=(10, 5))
sns.boxplot(data=unit_boxplot_data)
plt.title("Distribution of Units Sold by Month")
plt.xlabel("Month")
plt.ylabel("Units")
plt.show()

sales_boxplot_data = pd.DataFrame({
    "October": df_oct["Sales"].reset_index(drop=True),
    "November": df_nov["Sales"].reset_index(drop=True),
    "December": df_dec["Sales"].reset_index(drop=True)
})
plt.figure(figsize=(10, 5))
sns.boxplot(data=sales_boxplot_data)
plt.title("Distribution of Sales by Month")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```





7. Explore Monthly Plots and Analysis

In [10]:
```
monthly_totals = pd.DataFrame({
    "Month": ["October", "November", "December"],
    "Unit": [df_oct["Unit"].sum(), df_nov["Unit"].sum(), df_dec["Unit"].sum()],
    "Sales": [df_oct["Sales"].sum(), df_nov["Sales"].sum(), df_dec["Sales"].sum(
}).set_index("Month")

print("Overall Unit and Sales figures:")
display(df[["Unit", "Sales"]].sum().to_frame("Total"))
print("Units sold by month:")
display(monthly_totals[["Unit"]])
print("Sales by month:")
display(monthly_totals[["Sales"]])

monthly_totals["Unit"].plot(kind="bar", title="Units Sold by Month", ylabel="Uni
plt.tight_layout()
plt.show()
monthly_totals["Sales"].plot(kind="bar", title="Sales by Month", ylabel="Sales",
```

```
plt.tight_layout()
plt.show()

daily_monthly = daily.copy()
daily_monthly["Month"] = daily_monthly["Date"].dt.month_name()
month_order = ["October", "November", "December"]
fig, axes = plt.subplots(2, 1, figsize=(14, 9), sharex=True)
for month in month_order:
    part = daily_monthly[daily_monthly["Month"] == month]
    axes[0].plot(part["Date"], part["Unit"], label=month)
    axes[1].plot(part["Date"], part["Sales"], label=month)
axes[0].set_title("Consolidated Three-Month Unit Plot")
axes[0].set_ylabel("Units")
axes[0].legend()
axes[1].set_title("Consolidated Three-Month Sales Plot")
axes[1].set_ylabel("Sales")
axes[1].set_xlabel("Date")
axes[1].legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Overall Unit and Sales figures:

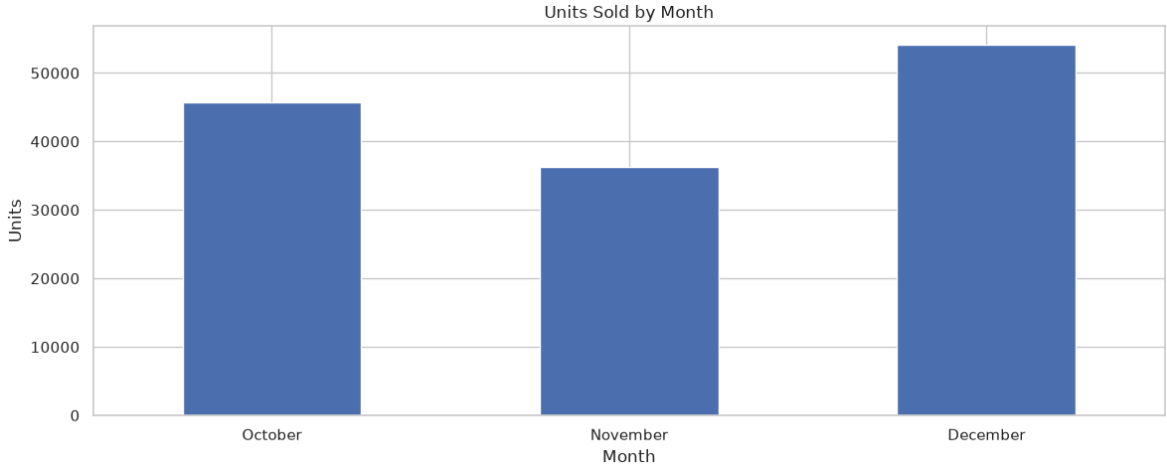
	Total
Unit	136121
Sales	340302500

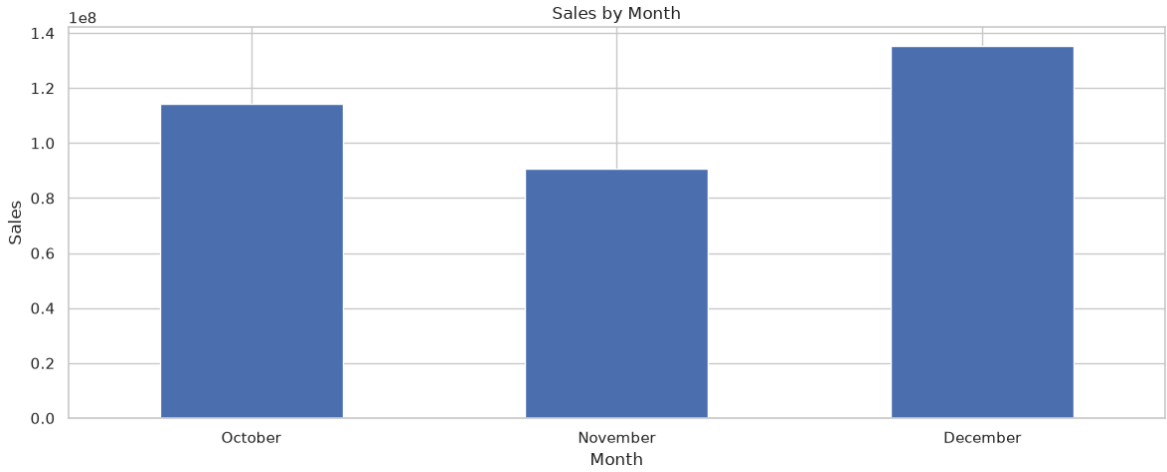
Units sold by month:

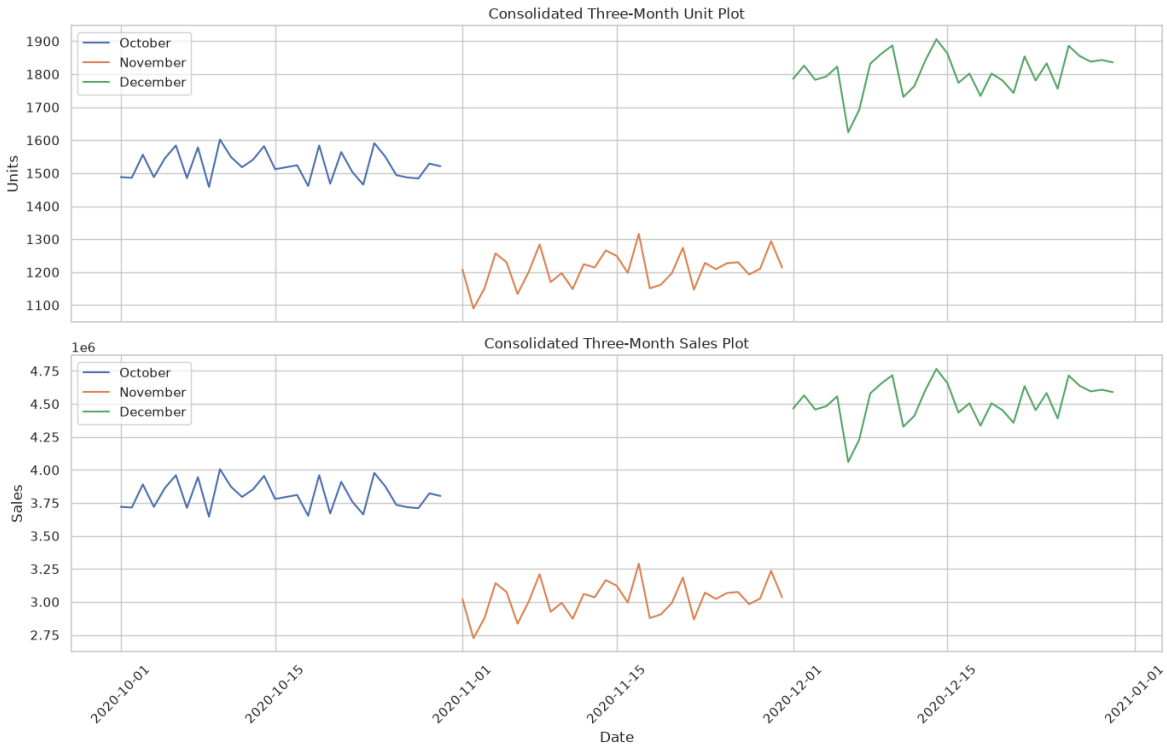
Month	Unit
October	45716
November	36273
December	54132

Sales by month:

Month	Sales
October	114290000
November	90682500
December	135330000







8. Obtain a Comprehensive Snapshot

In [11]:
```
print("Dataset dimensions:", df.shape)
print("Date range:", df["Date"].min().date(), "to", df["Date"].max().date())
print("Number of dates:", df["Date"].nunique())
print("Number of states:", df["State"].nunique())
```

```
print("Customer groups:", df["Group"].nunique())
print("Time categories:", df["Time"].nunique())
print("Total units:", df["Unit"].sum())
print("Total sales:", df["Sales"].sum())
display(df.describe(include="all").T)
```

```
Dataset dimensions: (7560, 6)
Date range: 2020-10-01 to 2020-12-30
Number of dates: 90
Number of states: 7
Customer groups: 4
Time categories: 3
Total units: 136121
Total sales: 340302500
```

	count	unique	top	freq	mean	min	25%	50%	75%
Date	7560	NaN	NaN	NaN	2020-11-15 04:00:00	2020-10-01 00:00:00	2020-10-23 00:00:00	2020-11-15 12:00:00	2020-12-08 00:00:00
Time	7560	3	Morning	2520	NaN	NaN	NaN	NaN	NaN
State	7560	7	WA	1080	NaN	NaN	NaN	NaN	NaN
Group	7560	4	Kids	1890	NaN	NaN	NaN	NaN	NaN
Unit	7560.0	NaN	NaN	NaN	18.005423	2.0	8.0	14.0	26.0
Sales	7560.0	NaN	NaN	NaN	45013.558201	5000.0	20000.0	35000.0	65000.0

9. Analyze Statewise Sales in the United States

In [ ]:
```
statewise_sales = df.groupby("State", as_index=False).agg(
    Unit=("Unit", "sum"), Sales=("Sales", "sum")
).sort_values("Sales", ascending=False)
display(statewise_sales)

statewise_sales.plot(x="State", y="Sales", kind="bar", legend=False,
                     title="Statewise Sales", ylabel="Sales", figsize=(14, 5))
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
statewise_sales.plot(x="State", y="Unit", kind="bar", legend=False,
                     title="Statewise Units Sold", ylabel="Units", figsize=(14,
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

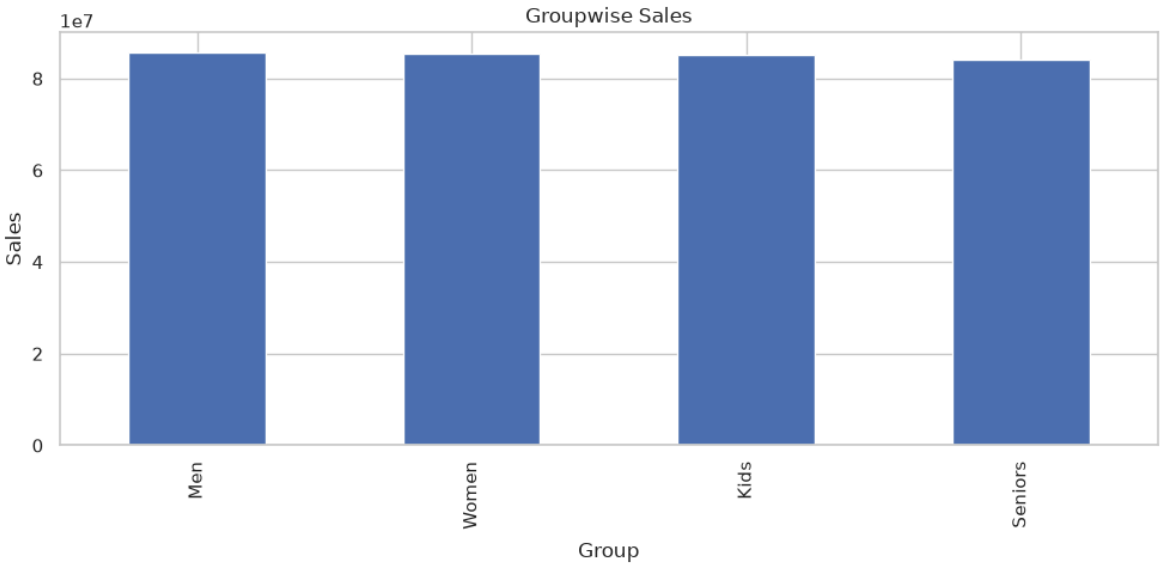
10. Conduct Groupwise Analysis

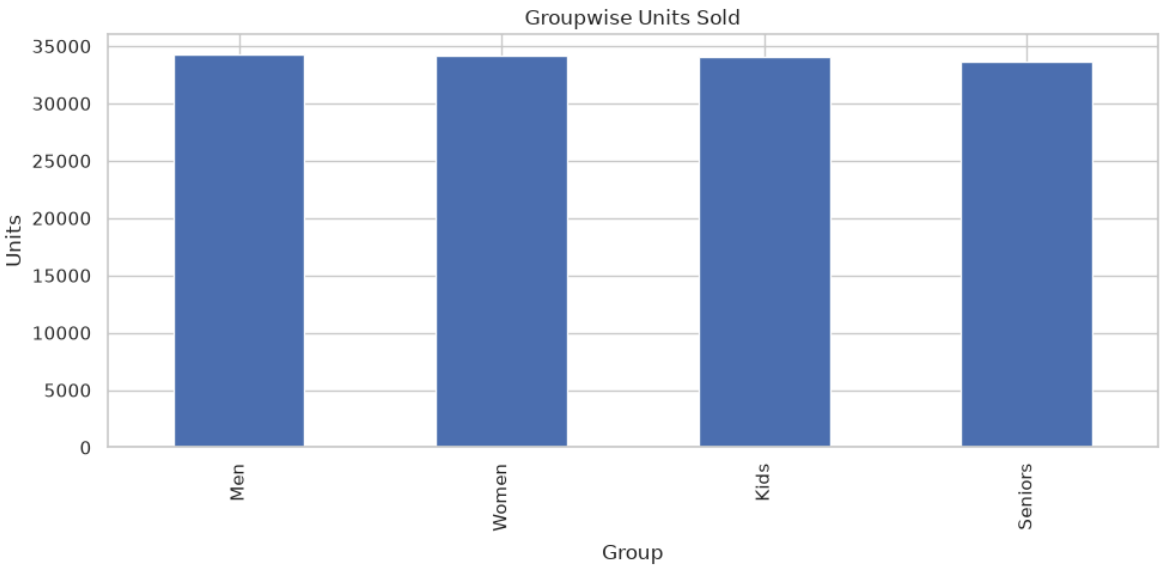
In [12]:
```
groupwise_sales = df.groupby("Group", as_index=False).agg(
    Unit=("Unit", "sum"), Sales=("Sales", "sum")
).sort_values("Sales", ascending=False)
display(groupwise_sales)

groupwise_sales.plot(x="Group", y="Sales", kind="bar", legend=False,
```

```
                    title="Groupwise Sales", ylabel="Sales", figsize=(10, 5))
plt.tight_layout()
plt.show()
groupwise_sales.plot(x="Group", y="Unit", kind="bar", legend=False,
                    title="Groupwise Units Sold", ylabel="Units", figsize=(10,
plt.tight_layout()
plt.show()
```

	Group	Unit	Sales
1	Men	34300	85750000
3	Women	34177	85442500
0	Kids	34029	85072500
2	Seniors	33615	84037500





11. Explore Timewise Analysis

In [13]:
```
timewise_sales = df.groupby("Time", as_index=False).agg(
    Unit=("Unit", "sum"), Sales=("Sales", "sum")
).sort_values("Sales", ascending=False)
display(timewise_sales)
```

```
timewise_sales.plot(x="Time", y="Sales", kind="bar", legend=False,
                    title="Timewise Sales", ylabel="Sales", figsize=(10, 5))
plt.tight_layout()
plt.show()
timewise_sales.plot(x="Time", y="Unit", kind="bar", legend=False,
                    title="Timewise Units Sold", ylabel="Units", figsize=(10, 5)
plt.tight_layout()
plt.show()
```

	Time	Unit	Sales
2	Morning	45683	114207500
0	Afternoon	45603	114007500
1	Evening	44835	112087500

